

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE: STATISTICS WITH R PROGRAMMING (ITA0404)

COURSE OUTCOME COVERED: CO3 -- ASSESSMENT TOOL 3 (ASSIGNMENT-2)

CO3: Construct and manipulate data frames using reshaping, merging, and file I/O operations in R to prepare structured datasets for analysis (K4)

Weightage: 30% | **Questions:** 3 | **Total Marks:** 30 | **Duration:** 1 Hour

Student Name: PAVITHRA.R

Register No: 192421369

Code of Conduct:

I PAVITHRA.R (192421369) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Reg. No: 192421369

RUBRICS FOR EVALUATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total Marks (30)
Conceptual Understanding	30				
Accuracy of Answer	25				
Application of Knowledge	20				
Completeness of Response	15				
Clarity & Presentation	10				
Total per Question	10 Marks	/10	/10	/10	/ 30

Q1. Analyze the Role of File Handling in R [10 Marks]

Company Scenario: A company receives employee information periodically in CSV files from different departments.

Question: Analyze how R file-handling operations can be used to manage this employee data. Explain the process of reading, inspecting, editing, validating, and writing the data back to a file. Discuss possible file handling problems and suggest suitable methods to overcome them.

1. Detailed File Handling Workflow

- **Reading:** Import department CSVs into R using `read.csv("dept_emp.csv", stringsAsFactors = FALSE)`.
- **Inspecting:** Utilize structural inspection tools like `str()`, `summary()`, `head()`, and `dim()` to evaluate missing values and variable data types.
- **Editing:** Clean inconsistent department names, handle missing values via `na.omit()` or imputation, and standardise column headers.
- **Validating:** Check for schema integrity (correct column count, unique employee IDs using `duplicated()`, and expected value ranges).
- **Writing:** Export clean aggregated datasets back to disk using `write.csv(clean_df, "master_employee_data.csv", row.names = FALSE)`.

2. File Handling Issues & Technical Solutions

- **Issue 1: Inconsistent File Formats or Column Headers:** Different departments may send CSVs with varying column titles.
Solution: Standardize column names dynamically using `colnames()` mapping or `dplyr::rename()` before merging.
- **Issue 2: Encoding / Special Character Errors:** Symbols (e.g., non-ASCII characters in employee names) become corrupted.
Solution: Explicitly specify encoding, e.g., `read.csv(..., fileEncoding = "UTF-8")`.
- **Issue 3: Missing or NA Value Propagation:** Operations break due to unhandled missing data.
Solution: Use `is.na()`, `complete.cases()`, or `na.rm = TRUE` in statistical summaries.

3. Practical R Demonstration Code

```
# Reading employee data
emp_data <- read.csv("dept_emp.csv", stringsAsFactors = FALSE, fileEncoding = "UTF-8")

# Inspection
cat("Dataset Dimensions:", dim(emp_data), "\n")
str(emp_data)

# Editing & Data Validation
emp_data$Department <- trimws(toupper(emp_data$Department)) # Standardize text
emp_data <- emp_data[!duplicated(emp_data$Emp_ID), ]          # Remove duplicates
emp_data$Salary[is.na(emp_data$Salary)] <- median(emp_data$Salary, na.rm = TRUE) # Impute NA

# Writing back to a clean file
write.csv(emp_data, "cleaned_master_employee.csv", row.names = FALSE)
```

Q2. Analyze Different File Reading and Writing Methods [10 Marks]

Organization Scenario: An organization needs to work with data stored in CSV, text, and R-specific files.

Question: Analyze and compare the different methods available in R for reading and writing files. Explain the purpose and suitable use of functions such as `read.csv()`, `read.table()`, `write.csv()`, and `write.table()`. Discuss which method would be most appropriate for maintaining a large tabular dataset and justify your choice.

1. Comparison Matrix of R File I/O Functions

Function	Primary Purpose	Default Separator	Best Suitable Use Case
<code>read.csv()</code>	Reads comma-separated values into a Data Frame	Comma <code>,</code>	Standard structured tabular data files exchangeable across tools.
<code>read.table()</code>	General file reader for text files with custom delimiters	Whitespace / Custom	Unstructured text files, TSV files, or custom tab/space delimited inputs.
<code>write.csv()</code>	Exports data frames to CSV format	Comma <code>,</code>	Generating portable reports readable by Excel, Python, and SQL databases.
<code>write.table()</code>	Writes data frames to text files with custom parameters	Whitespace / Custom	Flexible text file exports requiring specific delimiters or formatting options.

2. Best Method for Large Tabular Datasets & Justification

For handling and maintaining **large tabular datasets** (millions of rows), base functions like `read.csv()` or `read.table()` can become memory-intensive and slow.

- **Best High-Performance Alternative:** The `fread()` and `fwrite()` functions from the **data.table** package, or `saveRDS()` / `readRDS()` for native binary R storage.
- **Justification:**
 - **Speed & Memory Efficiency:** `data.table::fread()` automatically detects delimiters, headers, and column data types while executing in parallel multi-threaded CPU mode, making it up to 10x-20x faster than standard `read.csv()`.
 - **Preservation of Data Types:** Binary format (`.rds`) preserves exact R data types (factors, dates, structures) without re-parsing overhead.

3. Implementation Example

```
# Base R File I/O
base_data <- read.csv("large_dataset.csv", header = TRUE)
write.csv(base_data, "output_base.csv", row.names = FALSE)

# High-Performance File I/O for Large Datasets
library(data.table)
large_df <- fread("large_dataset.csv")      # Multi-threaded fast reader
fwrite(large_df, "output_fast.csv")        # Multi-threaded fast writer

# Native Binary Storage (Preserves Exact R Data Types)
saveRDS(large_df, "large_data.rds")
restored_df <- readRDS("large_data.rds")
```

Q3. Analyze Data Editing and File Management Workflow [10 Marks]

Data Analyst Scenario: A data analyst receives a raw customer data file containing missing values, incorrect entries, and unnecessary columns.

Question: Analyze the steps required to transform the raw file into a usable dataset using R. Explain how data can be read from the file, inspected, edited, validated, and written into a new file. Discuss why the original file should be preserved and how the modified file can be verified for correctness.

1. End-to-End Data Transformation Workflow

1. **Read Raw Data:** Import the dirty file using `read.csv("raw_customer_data.csv")`.
2. **Inspect Structure:** Run `head()`, `summary()`, and check missingness using `colSums(is.na(df))`.
3. **Edit & Clean:** Drop irrelevant columns, impute/remove NA values, fix incorrect entry types (e.g., convert text age to integer), and strip leading/trailing spaces.
4. **Validate:** Ensure logic rules hold true (e.g., Customer Age ≥ 0 , Purchase Amount ≥ 0).
5. **Export Clean File:** Save processed data to a new file
`write.csv(cleaned_df, "cleaned_customer_data.csv")`.

2. Preserving Original Raw Files & Verification Techniques

• Importance of Preserving the Raw File:

- **Auditability & Reproducibility:** Raw data serves as an immutable single source of truth. Maintaining untouched raw files ensures data pipelines can be re-run or audited at any point.
- **Error Recovery:** If a data transformation logic error occurs, preserving the original dataset guarantees no data loss and allows seamless script correction.

• Methods to Verify Modified File Correctness:

- **Row & Column Count Matching:** Verify dimension integrity using `dim(cleaned_df)`.
- **Statistical Range Checks:** Verify that numeric values fall within valid ranges using `summary(cleaned_df)`.
- **Assertion Testing:** Use unit assertion rules like `stopifnot(anyNA(cleaned_df$Customer_ID) == FALSE)` to programmatically guarantee quality before saving.

3. Complete R Script Transformation Workflow

```
# Step 1: Read raw customer dataset
raw_cust <- read.csv("raw_customer_data.csv", stringsAsFactors = FALSE)

# Step 2: Inspect raw dataset
cat("Raw Dimensions:", dim(raw_cust), "\n")
print(summary(raw_cust))

# Step 3: Edit - Drop useless columns, filter invalid values, trim whitespace
clean_cust <- raw_cust[, c("Customer_ID", "Name", "Age", "Total_Spent")]
clean_cust$Name <- trimws(clean_cust$Name)
clean_cust <- clean_cust[!is.na(clean_cust$Customer_ID), ]
clean_cust$Age[clean_cust$Age < 0 | clean_cust$Age > 120] <- NA # Fix outlier/incorrect entries
clean_cust$Age[is.na(clean_cust$Age)] <- median(clean_cust$Age, na.rm = TRUE)

# Step 4: Validate correctness
stopifnot(sum(is.na(clean_cust$Customer_ID)) == 0) # Asserts zero missing IDs
stopifnot(all(clean_cust$Age > 0)) # Asserts positive ages

# Step 5: Write into a new clean file (preserving original raw_customer_data.csv)
write.csv(clean_cust, "cleaned_customer_data.csv", row.names = FALSE)
cat("Data transformation workflow completed and verified successfully!\n")
```